MSC INFORMATION TECHNOLOGY · UNIVERSITY OF THE WEST OF SCOTLAND

MeetMind viva guides

What each part of MeetMind does, how it works, where the code is, and the questions examiners are likely to ask. Read "How MeetMind works" first, then your own chapter.

How MeetMind works

Everyone must know this

> You give MeetMind a meeting recording. It turns the speech into text, works out who said what, writes the minutes, and then checks every line of the minutes against what was actually said. Every point links back to the exact moment in the recording.

THE PROBLEM

AI summaries sometimes state things that were never said: a decision nobody made, or a task given to the wrong person. This is called **hallucination**. In meeting minutes it is dangerous, because people act on them. MeetMind's answer is to make the AI **show its evidence** for every point, then check that evidence automatically and let a person review it.

THE RESEARCH QUESTION

Does asking the AI to cite the transcript lines behind each point (configuration **C2, evidence-grounded**) reduce unsupported statements compared with an ordinary AI summary (configuration **C1, plain**)? Both use the same AI, the same transcript and the same output fields. The only difference is the citation instruction, so any difference in accuracy can be put down to grounding.

THE SIX PROCESSING STAGES

1. **Preprocessing** `0–5%`
   FFmpeg converts any upload to clean 16 kHz mono audio. *Emmanuel*

2. **Transcribing** `5–55%`
   Speech becomes timed words, using Whisper (local) or AssemblyAI (cloud). *Pascal*

3. **Diarizing** `55–75%`
   The voices are separated into Speaker 1, Speaker 2… (pyannote locally, AssemblyAI in the cloud). *Pascal*

4. **Aligning** `75–78%`
   Each word is given to a speaker and grouped into transcript lines. *Pascal*

5. **Summarizing** `78–93%`
   The AI writes the minutes, and every item cites the line numbers it came from. *Olayinka*

6. **Verifying** `93–100%`
   A second AI pass checks each item against its cited lines: supported, partial or unsupported. *Olayinka*

Everything is then saved in the database (*Kenneth*), shown in the web app (*Olayinka*) and can be exported or followed up (*Chinonye*). Emmanuel's tests and deployment keep it all running.

THE ARCHITECTURE IN ONE PARAGRAPH

The **web app** (React, which produces the HTML, with Tailwind CSS) runs in the browser. It talks to the **API**, a Python program built with FastAPI, over HTTP. Uploads are processed in a **background job**, so the user can leave the page, and progress is pushed to the browser over a WebSocket. Results go into a **database**

(SQLite, or PostgreSQL in the Docker set-up) through SQLAlchemy. The AI can run **locally** on a GPU (Whisper, pyannote, an Ollama language model) or in the **cloud** (AssemblyAI and Claude), chosen by one setting, `AI_PROVIDER`.

LOCAL VERSUS CLOUD, IN NUMBERS

An 88-minute recording took about **3 minutes** end to end in cloud mode: 5 s of preparation, 98 s of transcription, 45 s for the minutes and 18 s for the check. Local mode needs a powerful GPU and is much slower, but no audio leaves the server. That trade-off between **speed and privacy** is a good discussion point.

Questions any of you could be asked

What problem does MeetMind solve?

Writing minutes is slow, and AI summaries can contain statements nobody made. MeetMind produces minutes where every point is linked to, and checked against, the words actually spoken, so people can trust them or quickly see what to check.

What is new about it compared with Teams Copilot or Otter?

Those tools ask you to trust the summary. MeetMind makes the AI cite its sources, runs an independent check of every point, labels anything unsupported, and lets a person review the claims. We also measure whether grounding reduces hallucination (C1 versus C2).

How do you know it works?

Automated tests check the software. The evaluation measures transcription accuracy (word error rate), speaker accuracy (diarization error rate), summary accuracy (human labelling of C1 and C2 claims), how well the automatic check agrees with people, and usability (SUS questionnaire).

What are its limitations?

English only; recordings up to two hours; the automatic check is itself an AI and can be wrong, which is why people can review claims; in cloud mode, audio is processed by third parties; speaker separation struggles when people talk over each other.

What about ethics and privacy?

Recording cannot start until the user confirms everyone agreed; recordings stay private to the account and can be deleted; the processing location is disclosed; AI content is labelled; we obtained ethical approval before testing with people.

Did you use AI to build it?

Answer honestly, in line with the group's declaration: what tools were used, for what, and what the group itself decided, tested, evaluated and wrote. Then show you understand your part, which is what this guide is for.

Emmanuel Chieji `B01836233` Media processing, testing and deployment

> Your part is the front door. Whatever file someone uploads (a phone recording, a Zoom video, a browser recording) you turn it into one clean, standard audio file the AI can understand. You also make sure the whole system is tested, secure and online.

HOW MEDIA PROCESSING WORKS, STEP BY STEP

1. **Measure the length.** `probe_duration()` asks **ffprobe** for the file's duration. Browser recordings (WebM) are written as a stream and often have no duration in their header, so if ffprobe finds none, the file is decoded once with ffmpeg to measure it.
2. **Reject files that are too long.** Anything over the limit (2 hours by default, `max_duration_s`) is refused with a clear message, before any expensive work.

3. **Convert.** `to_wav()` runs **ffmpeg** with: `-vn` (drop any video), `-ac 1` (mono), `-ar 16000` (16 kHz), and `pcm_s16le` (uncompressed 16-bit WAV).
4. **Even out the volume** (local mode only) with `loudnorm`, the EBU R128 broadcast standard, so quiet and loud speakers are treated alike. Cloud mode skips it because it is the slowest part of the stage and AssemblyAI does its own.
5. **Compress for the cloud.** In cloud mode, `compress_for_upload()` makes a 48 kbps mono MP3: about 25 MB per hour instead of about 115 MB of WAV, so uploads are quicker.
6. **Report problems clearly.** Failures raise `PreprocessError` with a plain-English message, which Mindy shows to the user.

KEY FILES

- `backend/app/pipeline/preprocess.py`: conversion and duration checks
- `backend/app/pipeline/cloud_transcribe.py`: `compress_for_upload()`
- `backend/tests/` and `frontend/e2e/`: automated tests
- `.github/workflows/ci.yml`: automatic tests on GitHub (lint, type check, tests, build)
- `docker-compose.cloud.yml`, `deploy/cloud/Caddyfile`, `docs/DEPLOYMENT.md`: going live

A GOOD FIRST IMPROVEMENT

The upload page currently accepts MP3, WAV, M4A, MP4 and WebM (`ACCEPTED` in `frontend/src/pages/UploadPage.tsx`). Adding MOV and MKV, and testing them, is a small, real change you can show in the viva.

Practice viva questions

Why convert everything to 16 kHz mono?

Whisper and pyannote were trained on 16 kHz audio. Speech information sits mostly below 8 kHz, and by the Nyquist theorem 16 kHz sampling captures up to 8 kHz. Mono halves the data, and speakers are told apart by voice analysis, not by stereo channels.

What is the difference between ffprobe and ffmpeg?

ffprobe reads information about a file (length, format). ffmpeg converts it.

Why did browser recordings cause problems?

MediaRecorder writes WebM as a live stream, so the header often has no duration. We fall back to decoding the audio to measure its length.

Why check the length before converting?

To fail fast and cheaply. Converting or transcribing a 5-hour file only to reject it would waste minutes of processing and, in cloud mode, money.

What does continuous integration do here?

Every push to GitHub automatically runs linting, type checking, the backend and frontend tests, and a production build. A broken change is caught before it reaches the live server.

How are API keys protected?

They live only in a `.env` file on the server, which is excluded from Git. The browser never sees them: live captions use a 60-second token the server requests on the user's behalf.

Iloanugo Onyeka Pascal B01829049

Transcription, speakers and evaluation

> Your part turns sound into words and works out who said each word. You also lead the evaluation: proving with numbers how accurate MeetMind is, and whether evidence-grounding (C2) really beats the plain summary (C1).

TRANSCRIPTION (SPEECH TO TEXT)

- **Local:** `transcribe()` uses **faster-whisper**, a faster re-implementation of OpenAI's Whisper model. Settings: English only, **word timestamps** on (every word gets a start and end time), **voice activity detection** on (silence is skipped), and a **batched** mode on the GPU that decodes several speech regions at once.
- **Cloud:** `AssemblyAIClient.transcribe()` uploads the audio, starts a transcription job with `speaker_labels` on, then asks for the result every 3 seconds until it is complete. AssemblyAI returns words in milliseconds and speakers as letters (A, B…). `parse_transcript()` converts these to MeetMind's own format (seconds, `SPEAKER_00`, `SPEAKER_01`…), so every later stage works the same either way.

DIARIZATION (WHO SPOKE WHEN)

Locally, `diarize()` uses **pyannote.audio** in four steps: segmentation (where is speech), speaker counting, embeddings (a numeric "voice fingerprint" per segment) and clustering (grouping fingerprints that sound alike). It outputs turns such as "SPEAKER_00 from 0.0 to 12.4 s". It needs a Hugging Face token. Without one, all speech goes to one speaker and the user is told why.

ALIGNMENT (JOINING WORDS AND SPEAKERS)

`align()` gives each word to the speaker turn it overlaps most. A word in a gap goes to the nearest turn. Consecutive words by the same speaker are merged into one transcript line, but a new line starts after a pause of more than **2 seconds**, or when a line reaches **30 seconds**, which keeps citations precise.

REAL NAMES

- **Teams or Zoom transcript file (.vtt):** the most reliable source. Teams marks speakers as `<v Jane Doe>` and Zoom as `Jane Doe: text`. Those turns are used directly.
- **Suggestions:** a speaker who says "I'm Priya" is probably Priya. The AI can also suggest names, but only if it cites lines that contain the name. A list of words that are not names ("Sorry", "English", "Yoruba"…) prevents silly mistakes. Suggestions are **never applied automatically**: a wrong name attached to a task is worse than no name.

THE EVALUATION SCRIPTS ALREADY IN THE PROJECT

The `research/` folder already contains scripts. Your job is to understand them, run them on real data, extend them where needed and interpret the results:

SCRIPT	MEASURES
`ami.py`	Downloads the public AMI Meeting Corpus (20 test meetings with audio)
`e1_transcription.py`	Word error rate (WER) with jiwer, and speed
`e2_diarization.py`	Diarization error rate (DER), with a 0.25 s tolerance
`e3_summarization.py`	ROUGE and BERTScore against human-written AMI summaries
`e4_hallucination_sheet.py`	Builds a blind labelling sheet (models hidden as M1, M2), then scores it
`e5_speed.py`	Seconds per stage for 15, 30 and 60-minute meetings

Practice viva questions

What is word error rate?

(Substitutions + Deletions + Insertions) ÷ number of words in the correct transcript. 10% WER means roughly one word in ten is wrong. Lower is better. We lower-case and remove punctuation on both sides first, so formatting does not count as errors.

What is diarization error rate?

The share of time that is wrongly labelled: missed speech + false alarms + speaker confusion, divided by total speech time. The 0.25 s collar ignores small errors at the edges of turns.

Why are transcription and diarization separate steps?

They answer different questions ("what was said" and "who was speaking") and use different models. Alignment joins them using timestamps.

Why use the AMI corpus?

It is a standard, public research dataset of real meetings with correct transcripts, speaker labels and human summaries, so our results can be compared with published work, and it needs no new consent.

Why is the labelling blind?

If the labeller knew which output came from C2, they might judge it more kindly. Hiding the configuration behind codes removes that bias.

What is Cohen's kappa?

A measure of agreement between two labellers that corrects for agreement by chance. 1 is perfect, 0 is chance level. We use it to show the human labels are reliable, and to compare MeetMind's automatic check with the human labels.

Why offer both local and cloud transcription?

Local keeps audio private but needs an expensive GPU and is slower. Cloud runs on an ordinary server and is much faster, but audio is processed by a third party. Measuring both lets us discuss the trade-off with evidence.

Olayinka Williams Ajibola B01841833

Summaries, AI check, memory and UI/UX

> Your part is the brain and the face. You turn the transcript into minutes where every point cites its evidence, run the independent check, link meetings into a series, add live captions, and design what users see.

WRITING THE MINUTES (MAP-REDUCE)

1. **Clean and chunk.** Filler words ("um", "you know") and stutters ("the the") are removed from the text the AI sees only; the stored transcript keeps them. The transcript is split into chunks that fit the AI's context window, always at line boundaries, with 2 lines of overlap so nothing is lost at the edges.
2. **Map.** Each chunk is sent with lines numbered like `[42] Kenneth: …`. The AI returns partial notes as JSON: overview, key points, topics (with each speaker's position and the outcome), decisions, action items, open questions and risks. In C2, every item must list the line numbers that support it.
3. **Reduce.** The partial notes are merged into one final set, combining citations. If the merged notes are still too long, they are merged in groups first.
4. **Validate.** Every reply must match a strict JSON schema, checked with Pydantic. If it does not, the AI is asked again with the error message. Valid answers are cached, so a retried job resumes instead of starting over.
5. **Check citations exist.** Each item is marked *valid* (cites real lines), *missing* (cites nothing) or *invalid* (cites lines that do not exist).

C1 VERSUS C2: WHY THE COMPARISON IS FAIR

Both use the same AI, the same fields and the same transcript. C2's instructions add "use only information in the transcript" and "cite the line IDs". C1 does not see line numbers at all. Only grounding differs.

THE AI CHECK (VERIFICATION)

A separate pass shows each claim to the AI with **only the lines it cites**, plus one neighbouring line either side for context. The AI labels it **supported** (every detail is there), **partial** (gist right, a detail missing or different) or **unsupported**, with a reason of up to 20 words. If no AI is available, a transparent **word-overlap** check is used (60% or more of key words found counts as supported, 30% or more as partial), and the app says which method produced the labels.

SERIES MEMORY AND LIVE CAPTIONS

- **Series memory** (`pipeline/continuity.py`): when a meeting belongs to a series, its decisions are compared with earlier ones to flag **changes** (e.g. a launch date moved), and unfinished actions are checked for progress. The fallback is deliberately cautious, preferring no alert to a false one.
- **Live captions** (`routers/live.py` , `useLiveCaptions.ts` , `useRecorder.ts`): the browser converts the recording to 16 kHz audio with an AudioWorklet and streams it over a WebSocket to AssemblyAI. The server hands the browser a token that expires in 60 seconds, so the API key never leaves the server. Quick notes use keyword rules, which are instant and free.

KEY FILES

- `backend/app/pipeline/runner.py` : runs all six stages in order
- `backend/app/pipeline/summarize_llm.py` , `chunking.py` , `summary_schema.py` : the minutes
- `backend/app/pipeline/verify.py` : the AI check
- `backend/app/pipeline/llm_clients.py` : talking to Claude or Ollama
- `frontend/src/` : the web app (React, TypeScript, Tailwind), including Mindy (`components/Buddy.tsx`)

Practice viva questions

What is hallucination, and how does MeetMind reduce it?

An AI stating something not supported by its input. MeetMind tells the AI to use only the transcript, makes every item cite line IDs, flags citations to lines that do not exist, and then independently checks each item against the cited lines.

Why map-reduce rather than sending the whole transcript at once?

A long meeting can exceed the model's context window, and models lose detail in very long inputs. Summarising chunks and then merging them keeps detail and works for any length. The local model in particular has a small context window.

Can the AI check be wrong?

Yes, it is also an AI. That is why its method is shown, people can review every claim, and the evaluation measures how often it agrees with human labels.

Why force JSON output?

So the app can reliably read decisions, actions and citations as data and store them in the database. Free text would need fragile parsing.

Why not just trust a powerful model like Claude?

Even strong models can hallucinate, and users cannot see that from a fluent summary. Evidence links and checking make errors visible instead of hidden.

Why React instead of plain HTML and CSS?

React still produces HTML and CSS, but builds pages from reusable components and updates the screen as data changes (for example live progress and captions), which would be complex and error-prone by hand.

Oguanya Kenneth Ifeanyi B01843817

Database, accounts and group lead

> Your part is the memory and the locks. You look after how every meeting, transcript and summary is stored, how users sign up and log in safely, and how personal data is protected and deleted. You also lead the group.

THE DATABASE

MeetMind uses **SQLAlchemy** (Python classes that map to tables) with **SQLite** by default (one file), or **PostgreSQL** in the Docker set-up. The tables, in `backend/app/models.py`:

TABLE	HOLDS
`users`	Name, email, hashed password
`series`	Recurring meeting groups (belongs to a user)
`meetings`	Title, file, status, agenda, series (belongs to a user)
`jobs`	Processing progress, timings, errors (one per meeting)
`speakers`	Speaker labels, real names, name suggestions
`segments`	Transcript lines: time, speaker, text, original text if corrected
`summaries`	One per configuration (C1, C2): minutes, check results, series memory
`decisions`, `action_items`	Linked to a summary; actions have owner, deadline, status
`claim_reviews`	Human labels for the evaluation
`evaluation_runs`	Records of evaluation runs

Deleting: most links use `ON DELETE CASCADE`. Deleting a meeting removes its transcript, speakers, summaries, decisions and actions; deleting a user removes

all their meetings. Deleting a *series* uses `SET NULL`, so its meetings are kept.

Migrations: changes to the tables are written as **Alembic** migrations (`backend/alembic/versions/0001` to `0004`) and applied automatically when the server starts, keeping existing data. Back up the database before a migration.

ACCOUNTS AND SECURITY

- **Passwords** are hashed with **Argon2** (`security.py`) and never stored as text.
- **Login tokens** are **JWTs** signed with HS256 and the server's `SECRET_KEY`. They contain the user ID and an expiry (60 minutes by default).
- **Audio links** use a separate token that only works for one meeting's audio for 6 hours, because an HTML `<audio>` element cannot send login headers. Media tokens are refused as login tokens.
- **Same error message** for an unknown email and a wrong password, so attackers cannot discover which accounts exist.
- **Rate limiting** (`ratelimit.py`): 10 attempts per minute per IP address and per email, then HTTP 429 with a "Retry-After" time.
- **Sign-up code:** optional; compared with `hmac.compare_digest`, which takes the same time however many characters match, so the code cannot be guessed by timing.
- **Ownership:** every request checks the meeting belongs to the logged-in user (`deps.py`).

Practice viva questions

Why hash passwords instead of encrypting them?

Encryption can be reversed with the key; hashing cannot. At login we hash the attempt and compare. If the database leaked, the passwords would still not be readable.

Why Argon2 and not MD5 or SHA-256?

MD5 and SHA are designed to be fast, which helps attackers try billions of guesses. Argon2 is deliberately slow and memory-hungry and adds a random salt, and it won the Password Hashing Competition.

What is a JWT, and why does it expire?

A signed token that proves who you are without a session table. Tampering breaks the signature. Expiry limits the damage if a token is stolen.

What is a migration?

A versioned script that changes the database structure (for example, adding the series table) while keeping existing data, so every copy of the database can be upgraded the same way.

SQLite or PostgreSQL?

SQLite is one file, needs no server and suits a single-server pilot. PostgreSQL handles many simultaneous users and writes better, and is used in the Docker set-up. Your recommendation should explain when to switch.

What personal data does MeetMind hold, and how is it deleted?

Names, emails, hashed passwords, recordings and transcripts, which may contain anything said in a meeting. Deleting a meeting cascades to all of its data. Link this to UK GDPR principles: data minimisation, storage limitation and the right to erasure.

AS GROUP LEAD

Send agendas to the supervisor before meetings and keep minutes in the module's Project Process Documentation template. Record supervisor meetings in MeetMind and use the minutes it produces as your starting point: a nice demonstration of the product.

Chinonye Linda Nwanya B01841462 Exports, follow-up and usability

> Your part is what happens after the meeting. You make sure people can take the minutes away as Word, PDF or text files, send a follow-up email, and get calendar reminders for their tasks. You also find out, with real users, how easy MeetMind is to use.

EXPORTS

1. `build_minutes()` in `backend/app/export.py` gathers everything: overview, key points, topics with each speaker's position, decisions, action items (showing "NOT ASSIGNED" or "none stated" when the owner or deadline is unknown) and the transcript with times.
2. An **AI notice** goes at the top, and any point the check found only partly supported or unsupported is **flagged in the file itself**, so warnings are not lost when the minutes are shared.
3. Three formats: **TXT**; **Word** (python-docx); and **PDF** (WeasyPrint turns HTML into PDF, with fpdf2 as a fallback on Windows, where WeasyPrint's system libraries are often missing).

FOLLOW-UP TOOLS

- **Follow-up email** (`components/meeting/FollowUpTab.tsx`): builds a subject and body with decisions and actions. It opens in the user's email app when short enough; otherwise the user copies it, because very long `mailto:` links are cut off by some mail programs.
- **Deadlines** (`lib/deadlines.ts`): `parseDeadline()` turns "tomorrow", "next Friday", "end of the month" or "14 October" into a real date, counted from the day of the meeting. Vague deadlines ("ASAP", "soon") return nothing: MeetMind never guesses a date.
- **Calendar reminders:** `buildIcs()` creates an **.ics** file, the standard calendar format that Outlook, Google Calendar and Apple Calendar all import, with an all-day event and an alarm for each action.

USABILITY STUDY

The protocol, consent form and questionnaire are already drafted in `research/user_study/`, and `research/sus_score.py` calculates the scores. Review them, prepare the ethics application, then run the sessions once approval is granted.

How SUS is scored: 10 statements, each rated 1 to 5. For odd-numbered items, subtract 1 from the rating; for even-numbered items, subtract the rating from 5. Add them up and multiply by 2.5 to get a score from 0 to 100. About 68 is average; above 80 is very good.

Practice viva questions

Why flag unsupported points inside the exported file?

Exports are shared with people who never open MeetMind. If the warning were only on screen, the shared minutes would look fully trustworthy when they are not.

Why does MeetMind not guess vague deadlines?

"Soon" could mean tomorrow or next month. A guessed date could make someone miss a real deadline, or put a false one in their calendar. No date is safer than a wrong one.

What is an .ics file?

The iCalendar standard for calendar events. One file works in Outlook, Google Calendar and Apple Calendar, so we do not need to connect to each calendar service separately.

Why have a fallback PDF library?

WeasyPrint gives the best layout but needs system libraries that are often missing on Windows. The fallback means export never simply fails.

What is SUS, and why use it?

The System Usability Scale: a standard 10-question survey that is quick, free, widely used, and gives a 0–100 score you can compare with published averages.

Why only 5 to 8 participants?

Usability research (Nielsen) shows about five users reveal most usability problems. It is a qualitative check of usability, not a statistical survey.

What did participants agree to?

They read an information sheet, signed a consent form, could withdraw at any time, and their data was anonymised and stored securely, under the ethics approval.

How to use this guide

- Open your key files and match each step above to the code. You do not need to memorise code, only to explain what it does and why.
- Run MeetMind yourself and watch your stage happen in the progress panel.
- Practise the questions out loud with another member: one asks, one answers without reading.
- Make one real improvement in your own area. Changes you made yourself are the easiest to talk about confidently.

Prepared from the MeetMind source code as of October 2026. If the code changes, update the matching section.